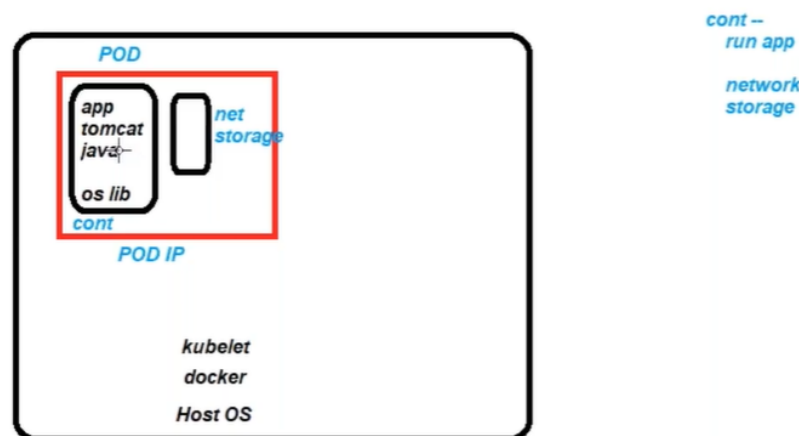


Kubernetes Objects/Resources

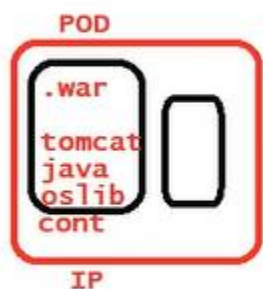
K8s is orchestrators for your application not for nodes, autoscaling for nodes then depend on cloud provider. K8s manages the container not the nodes.

Anything we create in kubernetes is called as Object ex: Pod, Replicaset, Deployment, Service etc...

The main objective of K8s is to deploy our application in the form of containers into set of machines that are configured as a worker node in the cluster however k8s doesn't directly deploy the containers on the worker node, **containers are encapsulated into k8s object called Pod**. If you need to scale your app, you can only do so by adding or removing pods. **The pod serves as a 'wrapper' for a single/multiple containers with the application code.**



Pod (Abstraction Over Container):



In K8s pod is the actual real thing, everything else revolve around, pod is smallest unit we created, pod is running instance of your application.

In k8s the only resource which physical run on a virtual machine are Pods, apart from that any other resource in k8s are just configuration they are not running executable.

Pod has ability to restart the container. In docker run time unit is container, In K8s run time unit is Pod. Pod is the smallest object in K8s.

How to deploy Pods?

Imperative approach: use the command line

kubectl run <name> --image=<name>

kubectl api-resource

kubectl explain pod

Declarative Approach?

Use config file to create resources (object) in cluster, using manifest. yaml

```
kind: Pod      ## kubectl api-resources
apiVersion: v1
metadata:
  name: pod2
  labels:      ## defining pod labels is must
    type: webserver
spec:
  containers:
    - name: cont1
      image: nginx
      ports:
        - name: http
          containerPort: 80
        - name: https
          containerPort: 443
    - name: cont2
      image: tomcat
      ports:
        - name: http
          containerPort: 8080
        - name: https
          containerPort: 8443
```

**Within the namespace pod name must be unique, may have same label name.
Pod controls the containers.**

In case of docker if container is stopped and restarted everything is changes e.g., it's IP, networking, in Case of Pod (**Abstraction layer over container just like tempered glass over the mobile**), **IP assigned to Pod not to the container**. In case of Pod if container restart inside the Pod it doesn't change the IP of Pod. Pod manage the container means containers are controlled by Pod, Pod as ability to restart the container in case container stopped. Only Pod and Node have IP addresses Container don't have.

In docker run time unit is container, while in K8s run time unit is Pod.

kind: What you are requesting k8s to create? Type of objects (Pod, Controller, Service...)

kubectl api-resources

apiVersion: Which version of the Kubernetes API you're using to create this object

kubectl explain <resource>

metadata: Information about the object (name, labels (alias)...differentiate between dev/prod env).

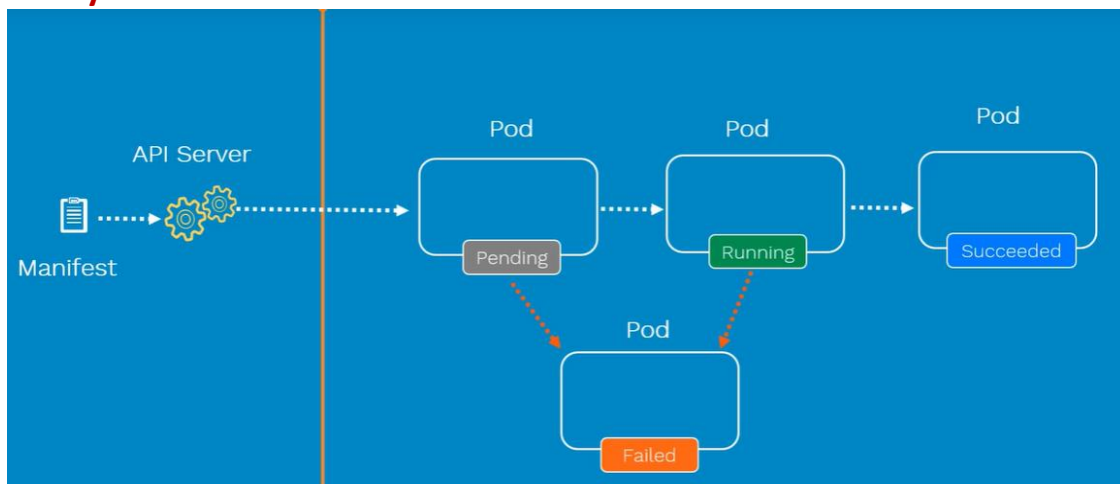
Data that helps uniquely identify the object, including a name string, UID, and optional namespace

spec: Specification what is inside object (Pod), Inside Pod we may have multiple containers

containerPort? Port at which container is running

status: not a mandatory, created by K8s by default.

Pod Lifecycle

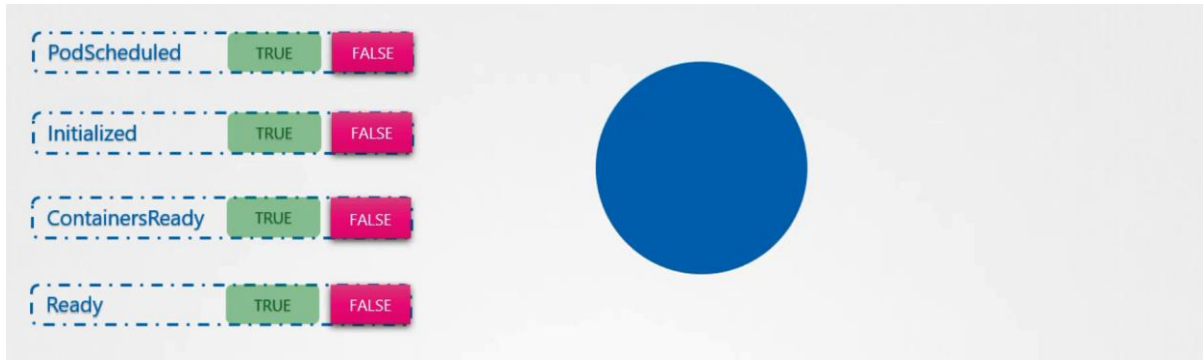


Pending: Pod Initialized and scheduler looking for appropriate Node

Container Creating: Pod scheduled on the node

Running: Once image pulled and container starts

Pod Conditions: complements Pod status



`kubectl describe pod <name>` // conditions section

Multi-Container Pod (Each container must have different image)

UC: In Docker nginx and splunk both are running as a different containers , splunk pulls the logs from nginx, In case nginx is down , Can splunk pull the logs? Even it runs can't pull the logs just waste of the resources.

key without car not of use.

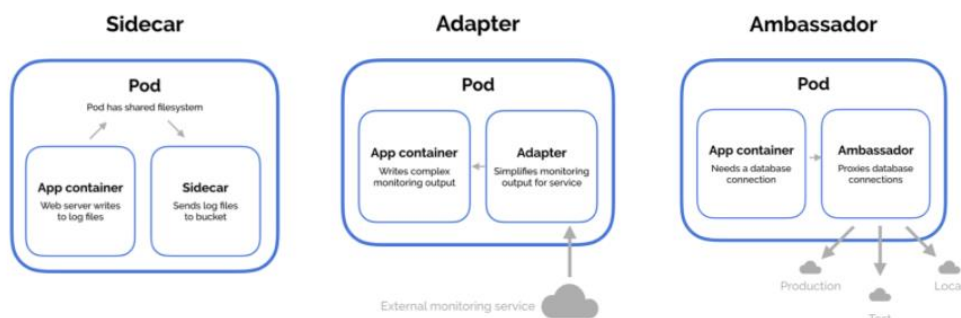
Multi Container Pods:

UC: we have two dependent microservice that share same set of resources like network, storage, rather than creating two different services, we can create both containers within same pod.

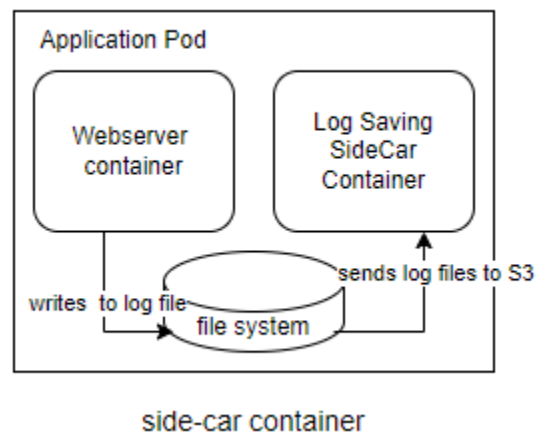
The Idea is decoupling of large monolith application in to small microservice and deploy as a small independent component.

Types of Multi Container Pods

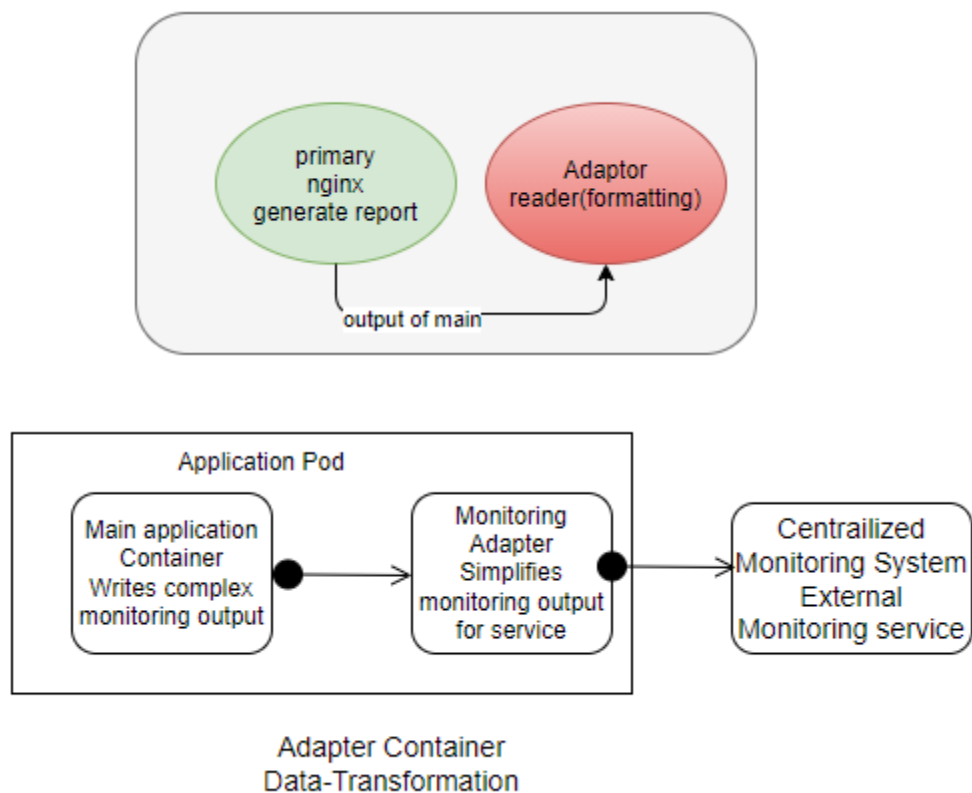
- Sidecar
- Adaptor
- Ambassador



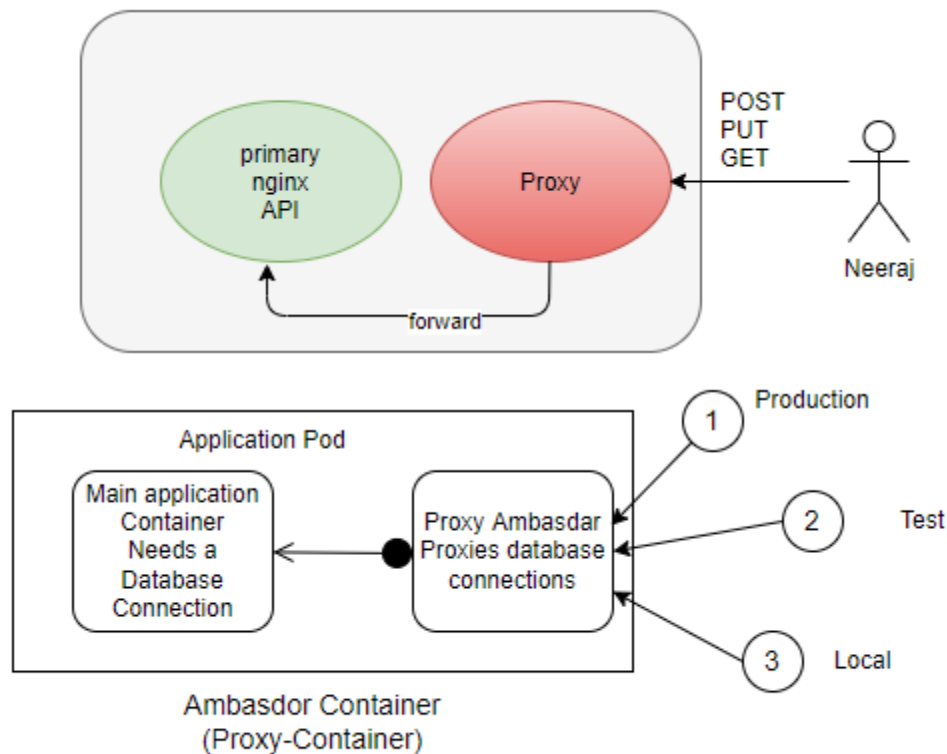
Sidecar:



Adapter: An adapter container transforms the **output of the main container**.
Example An adapter reads logs from container and transforms.



Ambassador: A HAProxy ambassador container receives network traffic and forwards it to the main container. Example an ambassador container listens on a custom port, and forwards the traffic to the main container's hard coded port.



In multi container pod second container (side container) running forever and in parallel with the main container.

How to access a specific container within the pod in case of multi container?

`kubectl exec -it <pod name> -c <container name> -- /bin/bash`

`kubectl exec multi-cont-pod -c cont3 -- ls -l`

```
kind: Pod          ### kubectl api-resources
apiVersion: v1     ### kubectl explain pod / deployment
metadata:
  name: mypod
  labels:
    app: web
    env: dev
spec:
  containers:
  - name: cont1
    image: nginx
    ports:
      - containerPort: 80
  - name: cont2
    image: tomcat
    ports:
      - containerPort: 8080
  - name: cont3
    image: httpd
    ports:
      - containerPort: 80
```

We can't run more than one container in same port within pod.

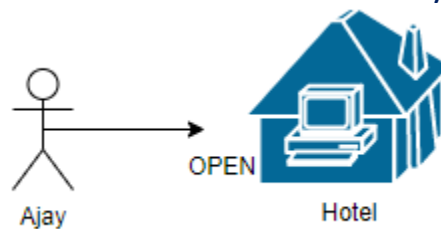
```

kind: Pod      ### kubectl api-resources
apiVersion: v1 ### kubectl explain pod / deployment
metadata:
  name: multi-cont-pod
  labels:
    app: web
    env: dev
spec:
  containers:
    - name: cont1
      image: nginx
      ports:
        - containerPort: 80
    - name: cont2
      image: tomcat
      ports:
        - containerPort: 8080
    - name: cont3
      image: lerndevops/samplepyapp:v1
      ports:
        - containerPort: 3000

```

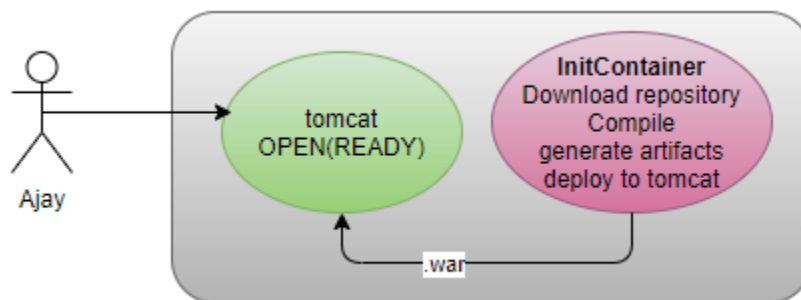
InitContainer Pod:

UC: User goes into hotel, but waiting for cleaning the hotel.
If hotel is ready, it does not mean food is also ready.



By default, when you run a Pod its show READY status **will not check container is responding or not**. If Pod is in READY status user access it, will it be response?

Solution: initContainer



Once initContainer completes only then status of the main container will be **READY**. Main Container waits till init container completes its activity.

```

Every 2.0s: kubectl get pods

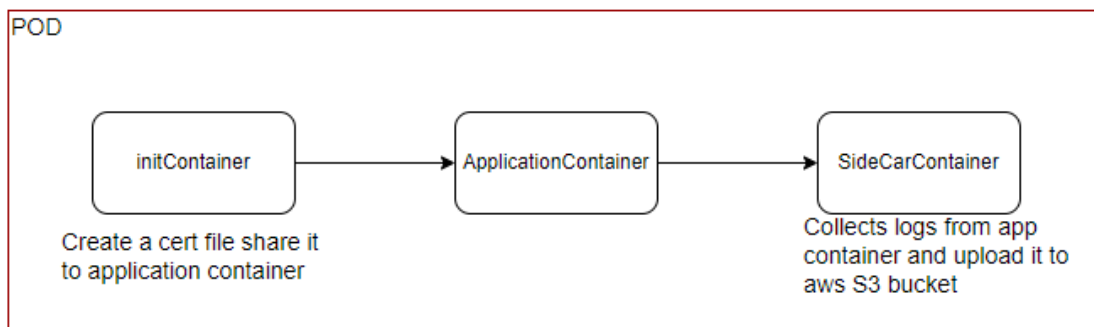
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-pod	0/1	Init:0/1	0	59s

```
Every 2.0s: kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-pod	1/1	Running	0	78s

InitContainer provides input to the main container, just like constructor parameter. In multi container pod second container running forever, while in init container which runs for completion, if init container successfully completed only then actual container will be start.



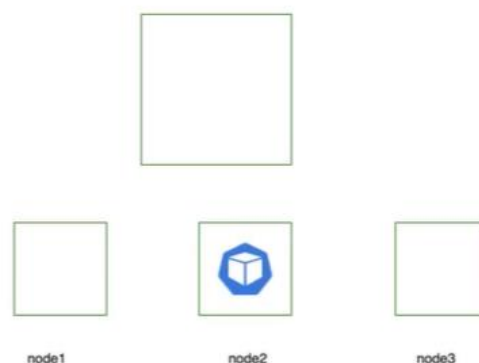
UC: We want to run Pod in a specific node? Scheduler not comes in picture

If we create a Pod in K8s, by default Pod can be go in any node.

There are two ways to run a Pod in a specific node

- Api-Server
- Directly Go to the Node and run the Pod there with the help of the kubelet (static pod).

YAML



Static Pods (Can't be scaled) *- Managed by kubelet

- Go to node
- Check the status of kubelet (service kubelet status)

- `/var/lib/kubelet/config.yaml` `# staticPodPath`

All master components run as a static pod.

kubelet check this location after every 3 seconds, if found any config file creates a resource.

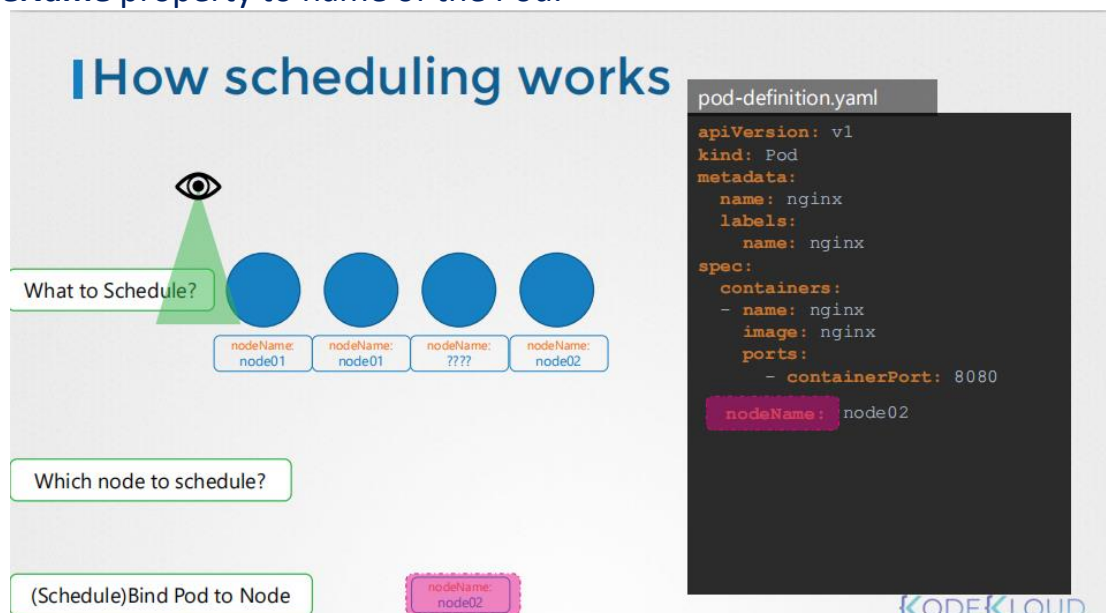
```
kubectl get Pods // visible to api-server  
kubectl delete pod --all //delete all pods
```

kubectl sends the update to api-server after every 5 seconds.

UC: run a Pod in specific node via from master (api-Server)

How Scheduling Works?

Every Pod have a field **nodeName** which is by default not set, scheduler goes to all pods and looks for those who don't have this property set, those are candidates for scheduling and then Identify the right node for those pods by running scheduling algorithm, once Identify schedule the pod on the node by setting the **nodeName** property to name of the Pod.



If there are no scheduler then Pod will be continuing on the pending state and you have to manually assign the pod to the node, without a scheduler simplest way to schedule the pod in node by assigning **nodeName** property of Pod to the name of the node. The pod is then assigned to the specified node. We can assign the nodeName at time of creation but what if Pod is already created? K8s don't allow to modify the nodeName property of the pod.

Scheduling Pods
deploy a pod without scheduler
spec:
 nodeName:
spec:
 nodeSelector:
 key: value

taint: condition that we apply on the node
toleration: condition that we apply on pod

By default, api-server takes help of scheduler to deploy the Pod, and scheduler selects the proper node and inform to api-server, then api-server interact with kubelet of the same node to deploy the pod.

Scheduler checks the following conditions before allocating the node

- H/W
- Node Status
- Pod definition- nodeName
- Labels/Selector
- Volume
- Taints and Tolerations
- Affinity check

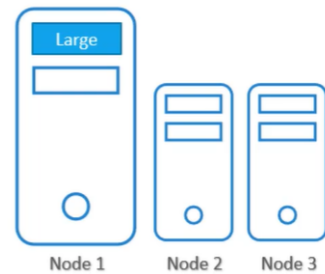
UC1: We directly convey the api-server, hey! api-server please deploy my pod in this specific node, using **nodeName** attribute of spec, scheduler not comes in picture.

Limitation: Single Condition, if specific node is busy or down, pod will be in Pending state.

UC2: We convey the criteria of the node to api-server, then api-server asks to scheduler, hey scheduler please schedule the Pod on the node have specific **labels** using the **nodeSelector** attribute of spec. Scheduler will assign the only nodes having specific labels, else pod will be in pending state.

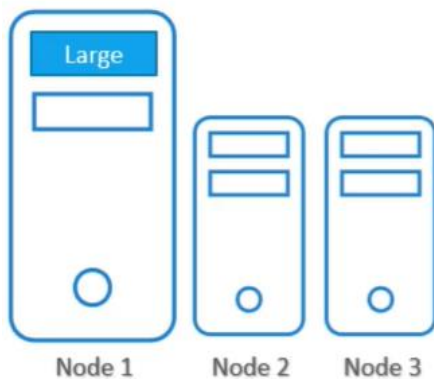
More Options available, there may be more than more nodes have same labels e.g., size=large.

```
pod-definition.yml
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: data-processor
    image: data-processor
  nodeSelector:
    size: Large
```



How we set node label e.g., size=Large?

```
kubectl label nodes <node-name> <label-key>=<label-value>
kubectl label nodes node-1 size=Large
```



Limitation of NodeSelector:

We can apply only equality criteria e.g., size =Large, but we can't define **complex criteria** e.g., Large or Medium, Not Small

Solution? Node Affinity

Node Affinity: Main purpose is that Pod is hosted on particular node.

How to add your own customized label to node?

```
kubectl describe node <worker2> // node has some labels
kubectl label node <worker2> env=prod
kubectl describe node <worker2>
kubectl get nodes --show-labels
```

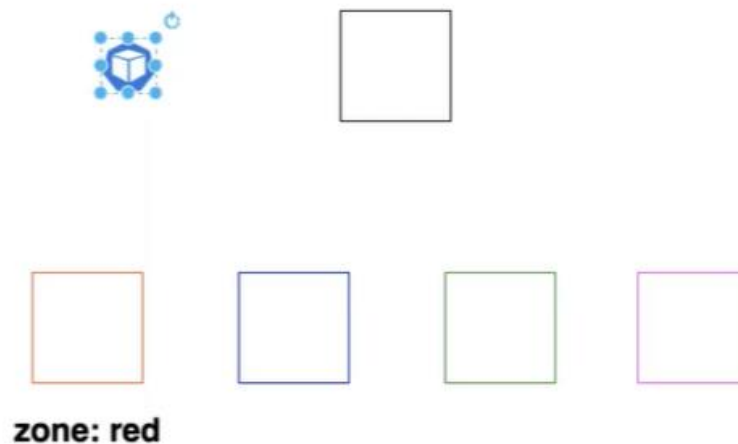
Who is checking the condition (Node or Pod)?

In above scenario decision authority is Pod.

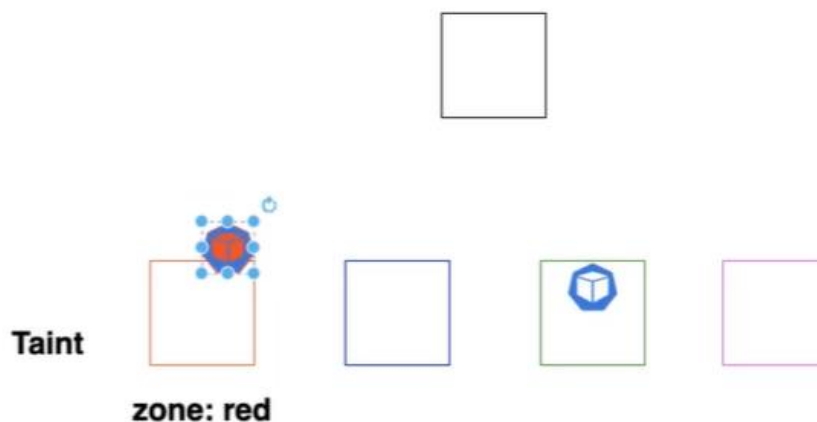
Taints (Node) & Toleration (Pod)

Taint: Condition Check at Node level, Decision power is with node.

UC3: Due to Covid Pandemic, Some Nodes are declared as a red zone, taint applied on the node. By default, by default no pod can be scheduled on the node in which taint is applied.



But even in pandemic, government employees, Dr. are allowed with their ID card, means if Pod have **toleration against the taint can be** scheduled on that node in which taint is applied.



How to set taints on Node?

```
kubectl taint nodes node-name key=value:taint-effect
```

NoSchedule | PreferNoSchedule | NoExecute

What happens to PODs
that DO NOT TOLERATE this taint?

- **NoSchedule:** Pod will not schedule on that Node.
- **PreferNoSchedule:** Scheduler will try to don't place the Pod but not guaranteed.
- **NoExecute:** New Pod will not be scheduled and existing Pod will be evicted if can't tolerant.

UC: Why scheduler don't place any pod on master node?

- Taint set on master with NoSchedule effect which don't allow to execute any Pod automatically till Pod has toleration against the taint.

How to set toleration to Pod against the taint on the node?

```
kubectl taint nodes node1 :
```

```
pod-definition.yml
apiVersion:
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: nginx-container
      image: nginx
  tolerations:
    - key: "app"
      operator: "Equal"
      value: "blue"
      effect: "NoSchedule"
```

Note: All values inside "".

Limitation: Pod have tolerations can be scheduled on the node in which taint is applied as well as also on other nodes.

Only Node has authority to allow the pod or not.

UC: We want to schedule specific pod in specific node.

Solution: Keep restriction on both, node (taint) and pod (affinity).

one container failure impacts the whole Pod, hence its suggested use only one container inside Pod.

logs functionality specific to the technology running container not to K8s.

Pod logs means container logs

Node logs means kubelet logs

Limits:

Containers are O/S level virtualization, they can consume memory or cpu they want to consume.

kubectrl run <name> --image=nginx --limits='cpu=.5, memory=200Mi' --request='cpu=.25, memory=100Mi'

```
root@masternode:~# kubectrl run limitspod --image tomcat --requests='cpu=0.3,memory=200Mi' --limits='cpu=0.5,memory=300Mi'
pod/limitspod created
root@masternode:~# kubectrl run limitspod --image tomcat --requests='cpu=0.3,memory=200Mi' --limits='cpu=0.5,memory=300Mi' --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: limitspod
  name: limitspod
spec:
  containers:
  - image: tomcat
    name: limitspod
    resources:
      limits:
        cpu: 500m
        memory: 300Mi
      requests:
        cpu: 300m
        memory: 200Mi
    dnsPolicy: ClusterFirst
    restartPolicy: Always
status: {}
root@masternode:~#
```

Environment Variables: To provide some data to your Pod explicitly

kubectrl run <name> --image=nginx --env='app=frontend' --env='env=dev'

kubectrl exec -it <name> -- bash

```
root@masternode:~# kubectrl run envpod --image nginx --env='app=abc' --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: envpod
  name: envpod
spec:
  containers:
  - env:
    - name: app
      value: abc
    image: nginx
    name: envpod
    resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Always
status: {}
root@masternode:~#
```

printenv

// print the environment variables inside container

```

root@envpod:/# printenv
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
HOSTNAME=envpod
PWD=/
app=abc
PKG_RELEASE=1-buster
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
NJS_VERSION=0.4.3
TERM=xterm
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PORT=443
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NGINX_VERSION=1.19.2
_=/usr/bin/printenv
root@envpod:/# echo $app
abc
root@envpod:/#

```

Probes:

Ready state of the Pod indicates that application inside the Pod is running and **ready to accept user traffic**. **How K8s know that application running inside the Pod is actually running or not.**

When you create a Pod and expose to external user using the service, service will route the traffic to the Pod immediately. The service relies on the Pod's Ready condition to route traffic. By default, K8s assumes as soon as pod is created is ready to serve user's traffic, so its sets the container's ready condition to true, but the application inside the container took longer to get ready, the service is unaware of that and sends traffic to the container by assuming that container is in ready state, causes user hit the service which is in not currently in ready state, we should wait till actual application inside the container ready to accept the traffic.

Application inside the container is ready?

Using: readinessProbe



When the container is created, K8s is not immediate set the Ready condition of the container is true, instead it performs the test to find that application response positively, and until that service is not forwarding any traffic to the Pod, as it sees that Pod is not ready.

```
readinessProbe:
  httpGet:
    path: /api/ready
    port: 8080
```

```
readinessProbe:
  tcpSocket:
    port: 3306
```

```
readinessProbe:
  exec:
    command:
      - cat
      - /app/is_ready
```

```
readinessProbe:
  httpGet:
    path: /api/ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 8
```

```
readinessProbe:
  tcpSocket:
    port: 3306
```

```
readinessProbe:
  exec:
    command:
      - cat
      - /app/is_ready
```

If readinessProbe failed request will not be send to Pod, status will be running but not READY to accept the request.

LivenessProbe:

UC: Application is not working but container is continuous staying alive e.g., due to bug in application. As per k8s concerns the container is UP so the application is to be assumed to be UP, but the user hitting the container is not served. In that case container need to be restart.

LivenessProbe configured on the container which periodically checks the health of the application

```
livenessProbe:
  httpGet:
    path: /api/healthy
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 8
```

```
livenessProbe:
  tcpSocket:
    port: 3306
```

```
livenessProbe:
  exec:
    command:
      - cat
      - /app/is_healthy
```

HTTP Test - /api/ready

TCP Test - 3306

Exec Command

If livenessProbe failed Pod will restart the container

Slightly impact the performance.

How team knows that pod is down?

Team don't worry, k8s auto healing feature will take care and will create a new pod in case of pod gets down, pods always run-on worker nodes.

If Pod Crash back-off??

Manually enable the Pod monitoring tool Prometheus. Whenever Pod status becomes Crash back-off it will automatically trigger an alert.

In cloud environment if any Pod comes down then cloud vendor will take care, in that case cloud controller concept comes in picture.

EKS-AWS

GKE- Google Cloud

kops: helps in cluster setup in AWS

UC1: Download file from S3

initContainer

UC2: Central Logging (Central Repository)

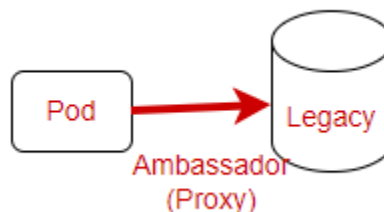
SideCarContainer- Will read the log and upload to S3 bucket

UC3: How to use vi, grep, commands in production, since we have light weight image in production environments

Using **Ephemeral** Containers (not defined in. yaml)

kubectrl debug

UC4: We have to call some Legacy code before actual call?



UC5: before calling transform from JSON to XML



annotations are used for third party tools and lib, not internal managed by K8s

```
kubectrl get pod -l 'env !=qa'
```

```
kubectrl get pod -l 'env notin (qa)'
```

```
kubectrl get pod -l 'env in (dev,prod)'
```

```
kubectrl get pod -l '!ingress'
```

```
kubectrl get pod -l 'env =dev, ingress=false'
```

```
kubectrl get pod -l 'env in (dev),ingress in (false)'
```